

PYTHON ESSENTIALS

Importing Libraries • Redirecting Output • Data Types

A hands-on guide for beginner and intermediate learners — with runnable programs and real output.

```
import this → "Simple is better than complex."
```

Agenda

01

Importing Libraries

Standard imports, aliases, selective imports — with examples

02

Redirecting Output

Sending `print()` to files, `sys.stdout`, and in-memory buffers

03

Data Types

Numbers, strings, booleans, collections — mutable vs immutable

04

Cheat Sheet & Takeaways

Quick reference tables to keep handy

Importing Libraries

Bring pre-written code — from Python's standard library or third parties — into your program.

```
import module
```

Loads the whole module; access with module.name

```
import module as alias
```

Loads the module under a short nickname

```
from module import name
```

Pulls one specific item into your namespace

```
from module import *
```

Pulls everything in (avoid — pollutes namespace)

```
concept_demo.py

# Beginner: load the whole module
import math
print(math.sqrt(16))

# Give it a short alias
import math as m
print(m.pi)

# Import specific names only
from math import sqrt, pi
print(sqrt(25), pi)

# --- Output ---
4.0
3.141592653589793
5.0 3.141592653589793
```

Example Program: Custom + Third-Party Modules

```
main.py

# file: shapes.py (a module you write)
def area_circle(r):
    return 3.14159 * r * r

# file: main.py
import shapes
import random

print(shapes.area_circle(4))
print(random.randint(1, 10))
```

```
terminal

--- Sample Output ---
50.26544
7
```

Beginner tip

- `random.randint(1, 10)` returns a new random integer each run — 7 is just one possibility.
- `shapes` is your own file; Python finds it because it sits in the same folder as `main.py`.
- Third-party libraries (e.g. `numpy`, `pandas`) need `pip` install first.

Redirecting Output

By default, `print()` writes to the console (`stdout`). You can send it somewhere else instead.

1

`print(..., file=f)`

Simplest way — pass an open file object

2

`sys.stdout = f`

Reroutes ALL future print calls at once

3

`contextlib.redirect_stdout`

Safely captures output for a limited block

`redirect_demo.py`

```
# Method 1: write straight to a file
with open('out.txt', 'w') as f:
    print('Hello, File!', file=f)
```

```
# Method 2: reroute sys.stdout
import sys
old = sys.stdout
sys.stdout = open('log.txt', 'w')
print('Logged line')
sys.stdout = old    # restore console
```

```
# --- out.txt contains ---
Hello, File!
```

Example: Capturing Output In Memory

capture.py

```
from contextlib import redirect_stdout
import io

buffer = io.StringIO()
with redirect_stdout(buffer):
    print('Step 1: loading data')
    print('Step 2: processing')
    for i in range(3):
        print('  item', i)

captured = buffer.getvalue()
print('Captured', len(captured), 'chars')
```

terminal

```
--- Console shows only ---
Captured 63 chars

--- Inside buffer.getvalue() ---
Step 1: loading data
Step 2: processing
  item 0
  item 1
  item 2
```

Python Data Types — Overview



Numeric

int, float, complex

```
age = 25  
price = 9.99  
z = 2+3j
```



Text

str

```
name = "Ada"
```



Boolean

bool

```
is_valid = True
```



Sequence

list, tuple, range

```
nums = [1, 2, 3]  
pt = (4, 5)
```



Mapping

dict

```
user = {"id": 1}
```



Set

set, frozenset

```
tags = {"a", "b"}
```



Binary

bytes, bytearray

```
data = b"hi"
```



None

NoneType

```
result = None
```

Mutable vs Immutable (for moderate learners)

Type	Mutable?	Example	Can change after creation?
int / float / bool	No	<code>x = 5</code>	<code>x = 6</code> creates a new object
str	No	<code>s = "hi"</code>	<code>s[0] = 'H'</code> raises an error
tuple	No	<code>t = (1, 2)</code>	Fixed size and contents
list	Yes	<code>L = [1, 2]</code>	<code>L.append(3)</code> changes it in place
dict	Yes	<code>d = {"a": 1}</code>	<code>d["a"] = 2</code> updates in place
set	Yes	<code>s = {1, 2}</code>	<code>s.add(3)</code> changes it in place

Rule of thumb: if a type has methods that modify it directly (append, add, update, pop, sort), it's mutable. Immutable types always return a brand-new object instead.

Example Program: Inspecting Every Data Type

data_types.py

```
a = 10                # int
b = 3.14              # float
c = 2 + 3j            # complex
d = "Hello"           # str
e = True              # bool
f = [1, 2, 3]         # list
g = (1, 2, 3)         # tuple
h = {"key": "value"}  # dict
i = {1, 2, 3}         # set
j = None              # NoneType

for v in [a, b, c, d, e, f, g, h, i, j]:
    print(v, '->', type(v).__name__)
```

terminal

```
--- Output ---
10 -> int
3.14 -> float
(2+3j) -> complex
Hello -> str
True -> bool
[1, 2, 3] -> list
(1, 2, 3) -> tuple
{'key': 'value'} -> dict
{1, 2, 3} -> set
None -> NoneType
```

Quick Reference Cheat Sheet

Topic	Syntax	Notes
Import module	<code>import os</code>	Access via <code>os.getcwd()</code> etc.
Import with alias	<code>import numpy as np</code>	Shorter name for frequent use
Import specific names	<code>from math import pi</code>	Use <code>pi</code> directly, no prefix
Print to file	<code>print(x, file=f)</code>	<code>f</code> must be an open file object
Redirect all stdout	<code>sys.stdout = f</code>	Remember to restore it after
Capture output safely	<code>redirect_stdout(buffer)</code>	Best for temporary capture
Check a type	<code>type(x)</code>	Returns the class, e.g. <code><class 'int'></code>
Immutable types	<code>int, float, str, tuple, bool</code>	Reassignment creates a new object
Mutable types	<code>list, dict, set</code>	Can be changed in place

Key Takeaways

1

Importing

`import`, `import ... as`, and `from ... import` give you three ways to bring in code — pick the clearest for your use case.

2

Redirecting output

`print()` can target any file-like object: real files, `sys.stdout`, or an in-memory `io.StringIO` buffer.

3

Data types

Every value has a type; know which ones are mutable so you don't get surprised by in-place changes.

Practice: re-type every example yourself — muscle memory beats memorizing.